

Loop

Product & Technical Documentation

Crypto stock trading on Arc Testnet

loopfiapp.xyz

Loop — Product & Technical Documentation

Version: 1.0

Last updated: June 2026

Live app: <https://loopfiapp.xyz>

Repository: github.com/Samped/loop

Table of contents

1. [What is Loop?](#)
2. [Who is it for?](#)
3. [Core concepts](#)
4. [Features walkthrough](#)

5. [How trading works](#)
 6. [System architecture](#)
 7. [Smart contracts](#)
 8. [Data sources & market intelligence](#)
 9. [Web application](#)
 10. [API reference](#)
 11. [Environment variables](#)
 12. [Local development](#)
 13. [Deployment \(Vercel & custom domain\)](#)
 14. [Security model](#)
 15. [Glossary](#)
 16. [FAQ & troubleshooting](#)
-

1. What is Loop?

Loop is a full-stack web application for trading **SoSoValue-indexed crypto stocks** on **Arc Testnet**. It combines live market data, news, and portfolio views with on-chain settlement in **USDC**.

Loop does **not** custody real-world equity tokens. Instead, it offers:

Product	Description
Synthetic spot	Buy and sell synthetic shares through a USDC-collateralized vault (<code>StockVault.sol</code>). Prices follow external oracles; settlement is in USDC.
Perpetual futures	Open leveraged long or short positions on selected tickers via <code>PerpEngine.sol</code> . Positions are cash-settled against mark prices.
Market intelligence	Sector views, live prices, charts, and aggregated news from multiple providers.
Portfolio	Unified view of wallet USDC, spot holdings, open perp positions, and closed trade history.

The app is built for **testnet experimentation**. Contracts, keys, and operational flows should be audited before any mainnet use.

2. Who is it for?

- **Traders and builders** exploring synthetic equity exposure on a USDC-native chain.
- **Developers** who want a reference implementation of oracle-driven spot vaults and perp engines with a modern Next.js frontend.
- **Teams** evaluating Arc Testnet for DeFi applications that settle in USDC.

You need a Web3 wallet (e.g. MetaMask), Arc Testnet configured, and testnet USDC from the [Circle Faucet](#).

3. Core concepts

Crypto stocks (SoSoValue index)

Loop lists instruments that track companies commonly held in crypto treasury or mining portfolios (e.g. MSTR, COIN, MARA). **SoSoValue** provides the index catalog, snapshots, klines, and reference prices. The app can list **100+ tickers** when `SOSOVALUE_API_KEY` is configured.

Synthetic vs tokenized equity

Loop uses **synthetic, cash-settled** exposure:

- **Spot:** The vault holds USDC reserves. When you buy, USDC is locked and you receive ledger shares at the oracle price. When you sell, shares are burned and USDC is returned.
- **Perps:** You post USDC margin. P&L is computed against mark prices; no shares change hands on-chain.

This is **not** wrapped stock tokens or direct equity ownership.

Arc Testnet

Property	Value
Network	Arc Testnet
Chain ID	5042002

RPC	<code>https://rpc.testnet.arc.network</code>
Explorer	testnet.arcscan.app
Gas token	USDC (native gas)

Oracle prices

On-chain contracts read prices from a designated **oracle address**. The Loop server (using `PRIVATE_KEY`) periodically pushes SoSoValue (spot) and computed mark (perp) prices to the contracts. Users' trades execute at the prices stored on-chain at transaction time.

4. Features walkthrough

Markets dashboard (/)

- Browse crypto stocks by sector.
- See live or cached snapshots (price, change, volume).
- Navigate to individual stock detail pages.
- On cold start, the server prefetches the full catalog in parallel so the list loads in seconds on production.

Stock detail (/stock/[ticker])

- Price chart (klines from SoSoValue).
- Company overview and key stats.
- **Trade panel:** connect wallet, buy/sell synthetic shares via `StockVault`.
- Ticker-specific news.

Perpetual markets (/perp , /perp/[ticker])

Supported perp tickers (configured at deploy time):

MSTR, COIN, HOOD, MARA, RIOT

- Live mark chart (server-side mark engine).
- Open long/short with leverage up to market limits.
- Add/remove margin, close positions.
- Funding and liquidation follow `PerpEngine` rules.

Portfolio (/portfolio)

- USDC balance on Arc.
- Spot holdings per ticker.
- Open perp positions with unrealized P&L.
- Closed trade history (stored locally and optionally in Neon Postgres).

News (/news , /news/[id])

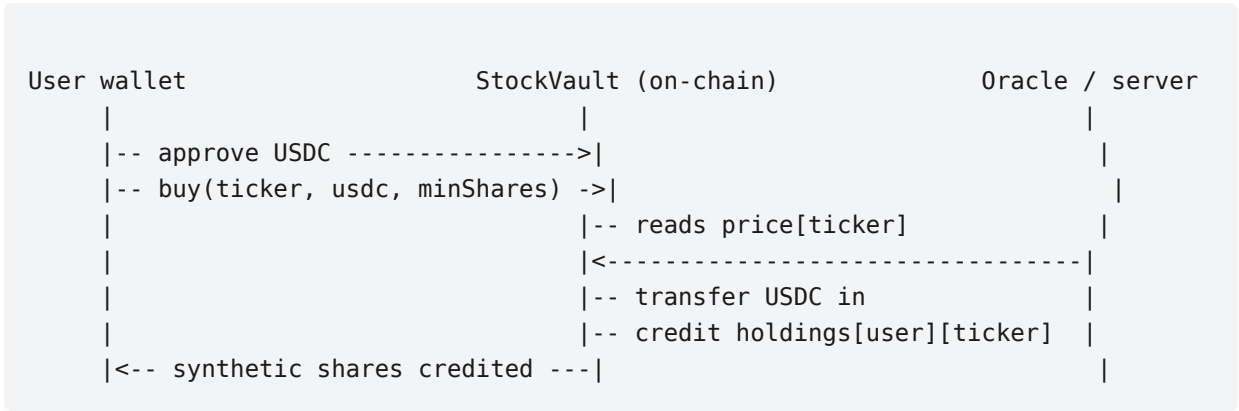
- Headlines from SoSoValue, Finnhub, CryptoPanic, and RSS fallbacks.
- Background sync and client polling keep the feed fresh.
- Full article view for stored items.

Agent (/agent)

- Placeholder for a future trading agent experience.

5. How trading works

Spot buy flow



1. User connects wallet and approves USDC spend.
2. User submits `buy` with USDC amount and slippage protection.
3. Contract uses the current oracle price to mint ledger shares.
4. Vault must remain **solvent**: $\text{USDC balance} \geq \text{sum of (circulating shares} \times \text{price)}$ for all tickers.

Spot sell flow

1. User submits `sell` with share amount and minimum USDC out.
2. Contract burns shares and transfers USDC from vault reserves.
3. Transaction reverts if reserves are insufficient.

Perp open flow

1. User selects side (long/short), size, and margin.
2. Contract checks leverage, open interest caps, and oracle freshness.
3. Position is stored; margin is locked in the engine.

Perp close / liquidation

- **Close:** User closes fully or partially; realized P&L and remaining margin are settled in USDC.
- **Liquidation:** Third parties can liquidate under-margined positions and receive a bonus from the insurance fund.

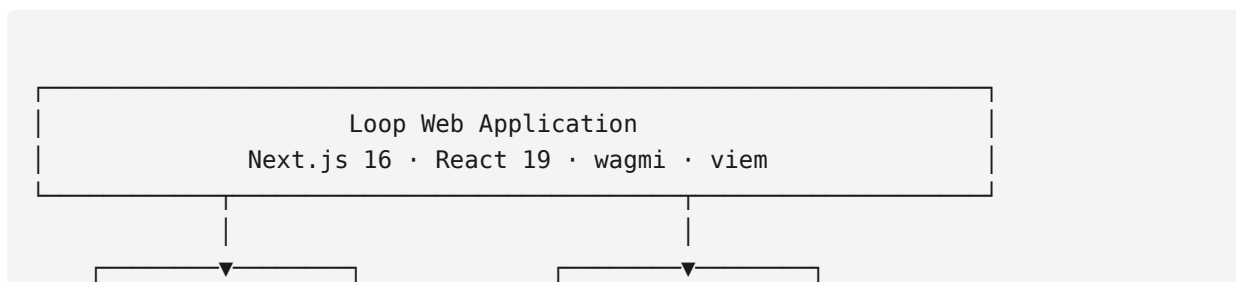
Mark price engine (perps)

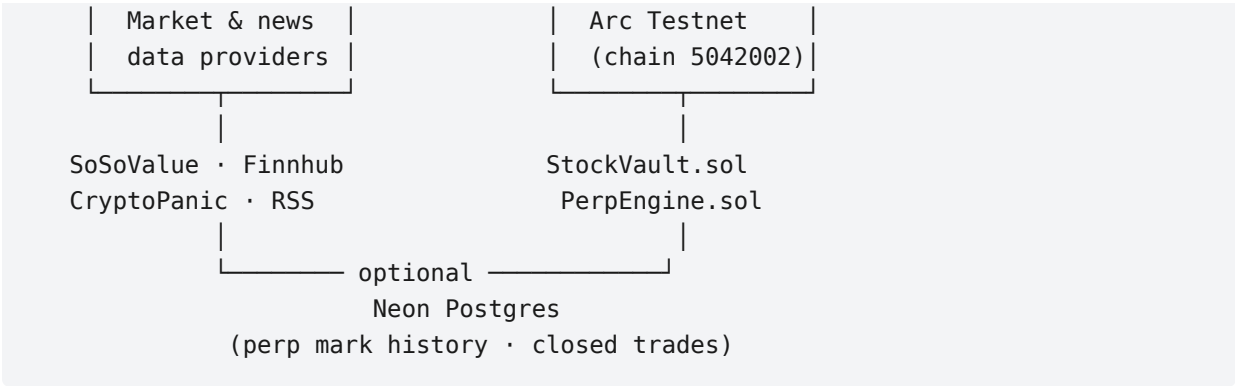
The server runs a **mark engine** (`PERP_MARK_MODE`):

Mode	Behavior
<code>live</code> (default)	Index from Finnhub + SoSoValue median; basis and microstructure model produce tradable marks.
<code>gbm</code>	Demo stochastic simulator for testnet demos without live feeds.

Marks are exposed read-only via `/api/perp/mark/[ticker]` and pushed to `PerpEngine` on an interval (`PERP_ORACLE_SYNC_INTERVAL_MS`).

6. System architecture





Layers

Layer	Role
Presentation	React pages and components; wallet connection via wagmi.
API routes	Next.js Route Handlers for market data, perp quotes, oracle sync, news.
Services (<code>src/lib</code>)	Caching, SoSoValue client, perp mark engine, news syncer, portfolio aggregation.
Persistence	In-memory and file caches; optional Neon for durable perp history.
Blockchain	viem clients with timeouts; user txs via wallet; server txs via <code>PRIVATE_KEY</code> .

Performance patterns

- **Cache-first APIs** return warm data immediately; background jobs refresh caches.
- **Neon circuit breaker** prevents database timeouts from blocking the server.
- **Parallel market prefetch** loads the full stock catalog on cold start (~8–12s on Vercel).
- **RPC timeouts** on Arc public RPC avoid hanging requests.

7. Smart contracts

Location: `contracts/` (Foundry)

StockVault.sol

USDC-reserved synthetic stock ledger.

Concept	Detail
Collateral	USDC (6 decimals)

Shares	18-decimal ledger units per ticker
Pricing	<code>prices[ticker]</code> in USDC per share (6 decimals)
Roles	<code>owner</code> , <code>oracle</code>
Key ops	<code>buy</code> , <code>sell</code> , <code>setPrice</code> , <code>depositReserve</code>

Solvency invariant: vault USDC must cover all outstanding synthetic liability at oracle prices.

PerpEngine.sol

Independent cash-settled perpetual futures.

Concept	Detail
Margin	USDC
Markets	Per-ticker config: leverage, maintenance margin, OI caps
Funding	8-hour intervals; cumulative funding index
Liquidation	Maintenance margin breach; liquidator bonus (2.5% bps config)
Oracle staleness	Max 5 minutes

Deploy scripts: `contracts/script/`

Tests: `forge test`

Deploy commands

Spot vault (from `web/`):

```
npm run deploy:vault
```

Perp engine (from `contracts/`):

```
forge script script/DeployPerp.s.sol --rpc-url https://rpc.testnet.arc.network --broadcast
```

Add deployed addresses to environment variables (see Section 11).

8. Data sources & market intelligence

Provider	Used for
SoSoValue	Stock catalog, snapshots, klines, primary news
Finnhub	Supplemental company news and perp index prices
CryptoPanic	Crypto-related headlines
RSS	Google News, CoinDesk fallbacks when API keys are absent

Caching

- Market snapshots and bootstrap payloads are cached server-side.
- News is ingested on a schedule (NEWS_SYNC_INTERVAL_MS) and on demand.
- Demo fallbacks apply when SOSOVALUE_API_KEY is missing (limited ticker set).

9. Web application

Tech stack

Layer	Technologies
Framework	Next.js 16 (App Router)
UI	React 19, Tailwind CSS 4
Wallet	wagmi 3, viem 2, TanStack Query
Database	Neon Postgres (optional)
Tooling	TypeScript, ESLint, tsx for scripts

Project structure

```
loop/
├── contracts/           # Solidity + Foundry
│   ├── src/
│   │   ├── StockVault.sol
│   │   └── PerpEngine.sol
│   └── script/
```

```

└─ web/
  └─ public/           # Logo, favicon, static assets
  └─ scripts/          # sync-prices, sync-news, deploy helpers
  └─ src/
    └─ app/            # Pages and API routes
    └─ components/     # UI components
    └─ hooks/          # Wallet, perp marks, balances
    └─ lib/            # Core business logic

```

Key libraries (`web/src/lib`)

Module	Purpose
<code>sosovalue.ts</code>	SoSoValue API client and parallel snapshot fetch
<code>market-cold-start.ts</code>	Full catalog prefetch on server boot
<code>perp-mark-engine.ts</code>	Live mark computation
<code>perp-mark-history-store.ts</code>	Mark history (local JSON + Neon)
<code>news-syncer.ts</code>	Multi-source news ingestion
<code>neon-guard.ts</code>	Circuit breaker for database calls
<code>arc-public-client.ts</code>	RPC client with timeout

10. API reference

Public read endpoints (representative list):

Endpoint	Method	Description
<code>/api/market/bootstrap</code>	GET	Initial market payload (sectors, stocks, snapshots)
<code>/api/market/stocks</code>	GET	Stock list
<code>/api/market/snapshots</code>	GET	All snapshots
<code>/api/market/snapshot/[ticker]</code>	GET	Single ticker snapshot
<code>/api/market/klines/[ticker]</code>	GET	Chart candles
<code>/api/market/prices/stream</code>	GET	SSE price stream
<code>/api/perp/mark/[ticker]</code>	GET	Current perp mark (read-only)

<code>/api/perp/quote/[ticker]</code>	GET	Open/close quote helpers
<code>/api/perp/positions/[address]</code>	GET	On-chain perp positions
<code>/api/portfolio/[address]</code>	GET	Aggregated portfolio
<code>/api/news</code>	GET	News list
<code>/api/news/article/[id]</code>	GET	Single article

Admin or rate-limited endpoints (require `x-admin-key` when `ADMIN_API_KEY` is set):

Endpoint	Method	Description
<code>/api/oracle/sync-prices</code>	POST	Push spot oracle prices
<code>/api/perp/sync-prices</code>	POST	Push perp mark prices
<code>/api/perp/liquidate</code>	POST	Trigger liquidation txs
<code>/api/news/sync</code>	POST	Run news ingestion

Nudge endpoints (`/api/oracle/nudge` , `/api/perp/nudge`) let the UI request a sync without exposing admin keys client-side; they are rate-limited server-side.

11. Environment variables

Copy `web/.env.example` to `web/.env.local` . Never commit secrets.

Market data

Variable	Required	Description
<code>SOSOVALUE_API_KEY</code>	Recommended	Live catalog and prices
<code>FINNHUB_API_KEY</code>	Optional	News and perp index
<code>CRYPTOPANIC_API_KEY</code>	Optional	Crypto news

Contracts

Variable	Description
<code>NEXT_PUBLIC_STOCK_VAULT_ADDRESS</code>	Spot vault (client)

STOCK_VAULT_ADDRESS	Spot vault (server oracle sync)
NEXT_PUBLIC_PERP_ENGINE_ADDRESS	Perp engine (client)
PERP_ENGINE_ADDRESS	Perp engine (server)
PRIVATE_KEY	Server wallet for deploy/sync only

Operations

Variable	Description
ADMIN_API_KEY	Protects admin POST routes
ORACLE_SYNC_INTERVAL_MS	Spot oracle auto-sync (default 60000)
NEWS_SYNC_INTERVAL_MS	News background sync (default 600000)
NEON_DATABASE_URL	Optional Postgres
NEON_ENABLED	Set <code>0</code> to disable Neon locally

Perp mark tuning

See `web/.env.example` for `PERP_MARK_MODE` , `PERP_INDEX_POLL_MS` , `PERP_ORACLE_SYNC_INTERVAL_MS` , and related parameters.

12. Local development

Prerequisites

- Node.js 20+
- npm
- Foundry (for contracts)
- Wallet with Arc Testnet + testnet USDC

Steps

```
git clone https://github.com/Samped/loop.git
cd loop/web
npm install
cp .env.example .env.local
```

```
# Edit .env.local with your keys and contract addresses
npm run dev
```

Open <http://localhost:3000>.

Useful scripts

Command	Description
<code>npm run dev</code>	Development server
<code>npm run build</code>	Production build
<code>npm run sync-prices</code>	Push spot prices to vault
<code>npm run sync-news</code>	Ingest news
<code>npm run warm-cache</code>	Prewarm snapshot cache
<code>npm run deploy:vault</code>	Deploy StockVault

Wallet setup

1. Add Arc Testnet: Chain ID `5042002` , RPC `https://rpc.testnet.arc.network` .
2. Get USDC from faucet.circle.com.
3. Connect in the app header.

13. Deployment (Vercel & custom domain)

Vercel

The app deploys from the `web/` directory.

1. Link the project to Vercel (`vercel.json` configures function timeouts).
2. Set root directory to `web` in project settings.
3. Add all production environment variables from Section 11.
4. Deploy; preview and production environments should both receive keys.

Required for full market catalog on production: `SOSOVALUE_API_KEY` , contract addresses, and optionally `NEON_DATABASE_URL` .

Custom domain (loopfiapp.xyz)

- Point apex domain **A record** to Vercel (`76.76.21.21`).
- Use `https://loopfiapp.xyz` (apex). If `www` is not configured in DNS, avoid apex → `www` redirects until `www` resolves.

Background jobs on serverless

Oracle sync, news sync, and mark engine ticks are started via `instrumentation.ts` on server boot. For always-on cron in production, consider Vercel Cron hitting nudge endpoints or an external worker.

14. Security model

Topic	Practice
Secrets	<code>PRIVATE_KEY</code> and <code>ADMIN_API_KEY</code> only on server; never in client bundles
Admin routes	Require <code>x-admin-key</code> header when <code>ADMIN_API_KEY</code> is set
Perp marks	Read-only from browser; engine runs server-side
Rate limits	Bootstrap, oracle sync, news sync, and nudge endpoints are rate-limited
CSP	Content Security Policy in <code>next.config.ts</code>
User funds	Users sign all trades with their own wallet; app does not custody keys

`PRIVATE_KEY` is used for **oracle price updates and operational txs**, not for user login or trading on behalf of users.

15. Glossary

Term	Definition
Crypto stock	Equity instrument tracked in the SoSoValue crypto stock index
Synthetic spot	Ledger shares in StockVault, backed by USDC reserves

Mark price	Perp reference price used for P&L and liquidation
Index price	External reference (e.g. median of feeds) underlying the mark
Funding	Periodic payment between longs and shorts on perps
Oracle	On-chain address authorized to set prices
Solvency	Vault USDC $\geq$ synthetic liabilities at oracle prices

16. FAQ & troubleshooting

Only a few stocks show on production

- Confirm `SOSOVALUE_API_KEY` is set in Vercel Production.
- Check `/api/market/bootstrap` response: `source` should be `sosovalue`, not `demo`.
- Cold start may take ~8–12 seconds for the full catalog.

Trades fail or prices look stale

- Run oracle sync: `npm run sync-prices` or POST `/api/oracle/sync-prices`.
- Verify `STOCK_VAULT_ADDRESS` / `PERP_ENGINE_ADDRESS` and `PRIVATE_KEY` on the server.
- Ensure oracle address on-chain matches the server wallet.

Database timeouts

- Set `NEON_ENABLED=0` locally if Neon is unreachable.
- Production uses a circuit breaker; mark history falls back to local JSON.

Perp page shows fewer than 5 markets

- Only **MSTR**, **COIN**, **HOOD**, **MARA**, **RIOT** are deployed on `PerpEngine`.
- Other tickers are spot-only unless new markets are configured on-chain.

News not updating

- Trigger sync via admin or wait for `NEWS_SYNC_INTERVAL_MS`.
- News page polls automatically; use Refresh if needed.

Document information

This document describes the Loop application as of June 2026. For the latest code-level details, see the repository README and inline source comments.

Support links

- [Arc documentation](#)
- [SoSoValue API](#)
- [Foundry book](#)
- [Circle Faucet](#)

Loop — Crypto stock trading on Arc Testnet.